

Ψηφιακή Επεξεργασία Εικόνας

Εργασία 2: Ανίχνευση ακμών και κύκλων

Στυλιανός Ανδρεάδης
ΑΕΜ: 10022

2.1 FIR filters

```
def fir_conv(  
    in_img_array: np.ndarray,  
    h: np.ndarray,  
    in_origin: np.ndarray,  
    mask_origin: np.ndarray  
) -> np.ndarray, np.ndarray:
```

Για την συνάρτηση του FIR υλοποιώ την συνέλιξη του πίνακα της εικόνας εισόδου με τον πίνακα h . Αρχικά, τα στοιχεία του πίνακα h εναλλάσσονται για να προσομοιώσουμε τη συνέλιξη $(x, y) \rightarrow (y, x)$, δηλαδή παίρνουμε τον ανάστροφο πίνακα h^T .

Στη συνέχεια, ανάλογα με το μέγεθος n του h^T εφαρμόζουμε την τεχνική του `zero padding` στον πίνακα της εικόνας εισόδου. Για παράδειγμα για `input` πίνακα 5×5 και πίνακα h 3×3 δημιουργούμε έναν νέο πίνακα 7×7 `padded_input` με μηδενικά στοιχεία στα κελιά που προστέθηκαν.

Τέλος, για κάθε στοιχείο του `input` παίρνουμε τον $n \times n$ πίνακα γύρω του και τον πολλαπλασιάζουμε στοιχείο-στοιχείο με τον h^T , αθροίζοντας τα επιμέρους στοιχεία και επαναλαμβάνοντας για κάθε στοιχείο του `input` δημιουργείται ο πίνακας εξόδου.

2.2 Τελεστής Sobel

```
def sobel_edge(  
    in_img_array: np.ndarray,  
    thres: float  
) -> np.ndarray:
```

Η συνάρτηση χρησιμοποιεί του δύο πίνακες που δίνονται και με τη βοήθεια της `fir_conv()` υπολογίζει το `gradient` για κάθε σημείο της εικόνας εισόδου.

Επιλέγοντας κατάλληλα το `threshold` λαμβάνουμε έναν πίνακα με (1) για ακμές και (0) αλλού.

2.3 Τελεστής Laplacian of Gaussian (LoG)

```
def log_edge(  
    in_img_array: np.ndarray  
) -> np.ndarray:
```

Και σε αυτή τη συνάρτηση χρησιμοποιούμε μια αρκετά straightforward προσέγγιση. Χρησιμοποιείται ο 5x5 πίνακας που δίνεται στην εκφώνηση και η συνάρτηση `fir_conv()`, που υλοποιήθηκε προηγουμένως.

Για κάθε σημείο του πίνακα που δημιουργείται “κοιτάμε” τους 8 γείτονες του

και βλέπουμε εάν το πρόσημο εναλλάσσεται, εάν συμβαίνει αυτό ανιχνεύουμε ακμή.

Για να αποφύγουμε το θόρυβο ελέγχουμε μόνο εσωτερικά στοιχεία του πίνακα και επίσης αγνοούμε αλλαγές προσήμου που δεν έχουν μεταξύ τους απόσταση μεγαλύτερη του `min_dif`.

2.4 Ανίχνευσης κυκλικών περιγραμμάτων με τη μέθοδο Hough

```
def circ_hough(  
    in_img_array: np.ndarray,  
    R_max: float,  
    dim: np.ndarray,  
    V_min: int  
) -> np.ndarray, np.ndarray:
```

Αρχικά δημιουργούμε έναν τρισδιάστατο πίνακα με διαστάσεις x , y , r και χωρίζουμε την κάθε διάσταση σε “περιοχές”.

Στην συνέχεια λαμβάνοντας τον πίνακα των 0 και 1 από κάποια από τις παραπάνω μεθόδους παίρνουμε τα σημεία ακμών και δοκιμάζοντας διαφορετικές γωνίες και ακτίνες βρίσκουμε πιθανά κέντρα κύκλων και ακτίνες.

Η διαδικασία επαναλαμβάνεται για όλα τα σημεία και τα πιθανά κέντρα-ακτίνες μαζεύουν ψήφους. Για όσα καταφέρουν να περάσουν ένα κατώτατο όριο που θέτουμε δημιουργούμε κύκλους.

Για να αποφύγουμε τον θόρυβο, χρησιμοποιούμε R_{min} , ώστε να μην εντοπίζονται μικροί κύκλοι πάνω σε ακμές.

- ➔ Επειδή η διαδικασία εύρεσης των κύκλων είναι πολύ χρονοβόρα και περίπλοκη σμικρύνουμε την εικόνα ώστε να έχει μέγιστη διάσταση 300 px και προσαρμόζουμε την άλλη ώστε να διατηρηθεί η αναλογία. Επίσης στην εύρεση των κέντρων ελέγχο για γωνίες 0-360 μοίρες με βήμα 30.

INPUT IMAGE



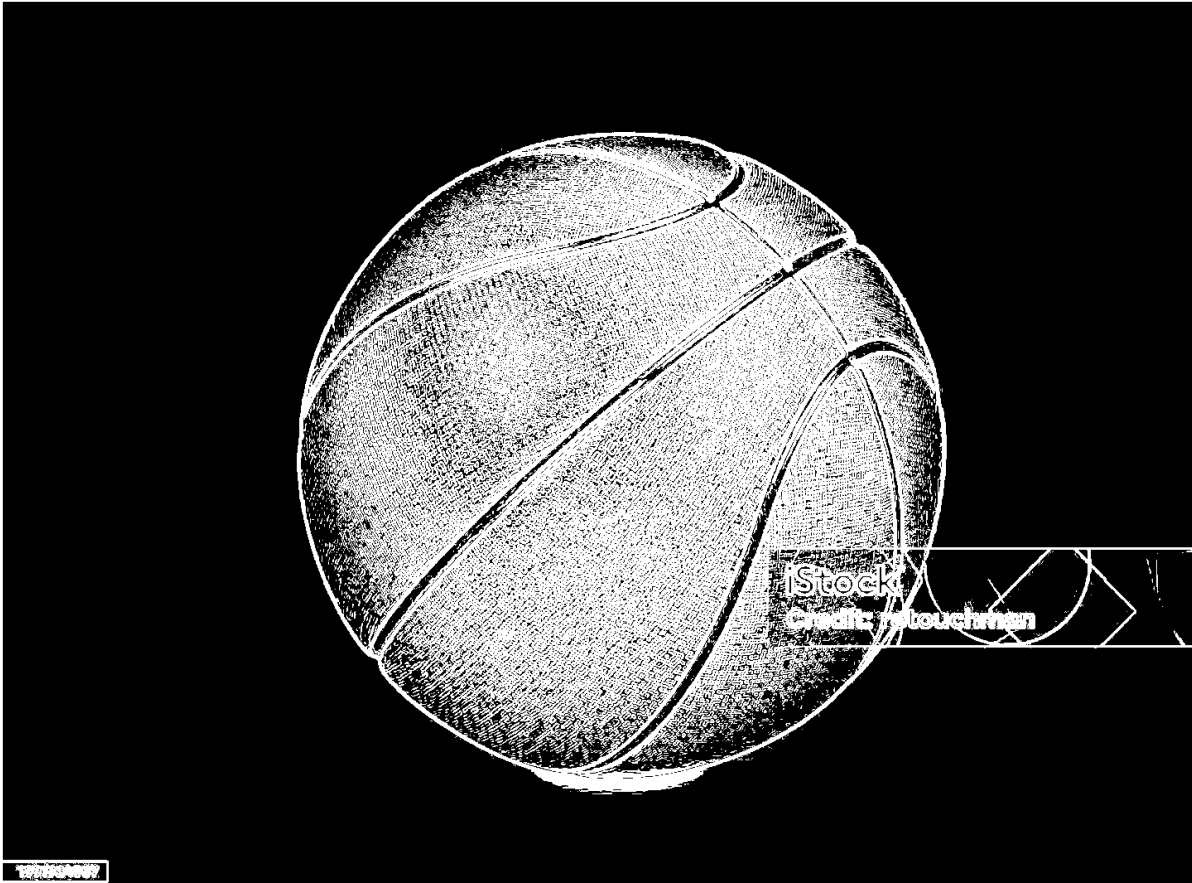
1991934907

INPUT GRAYSCALE

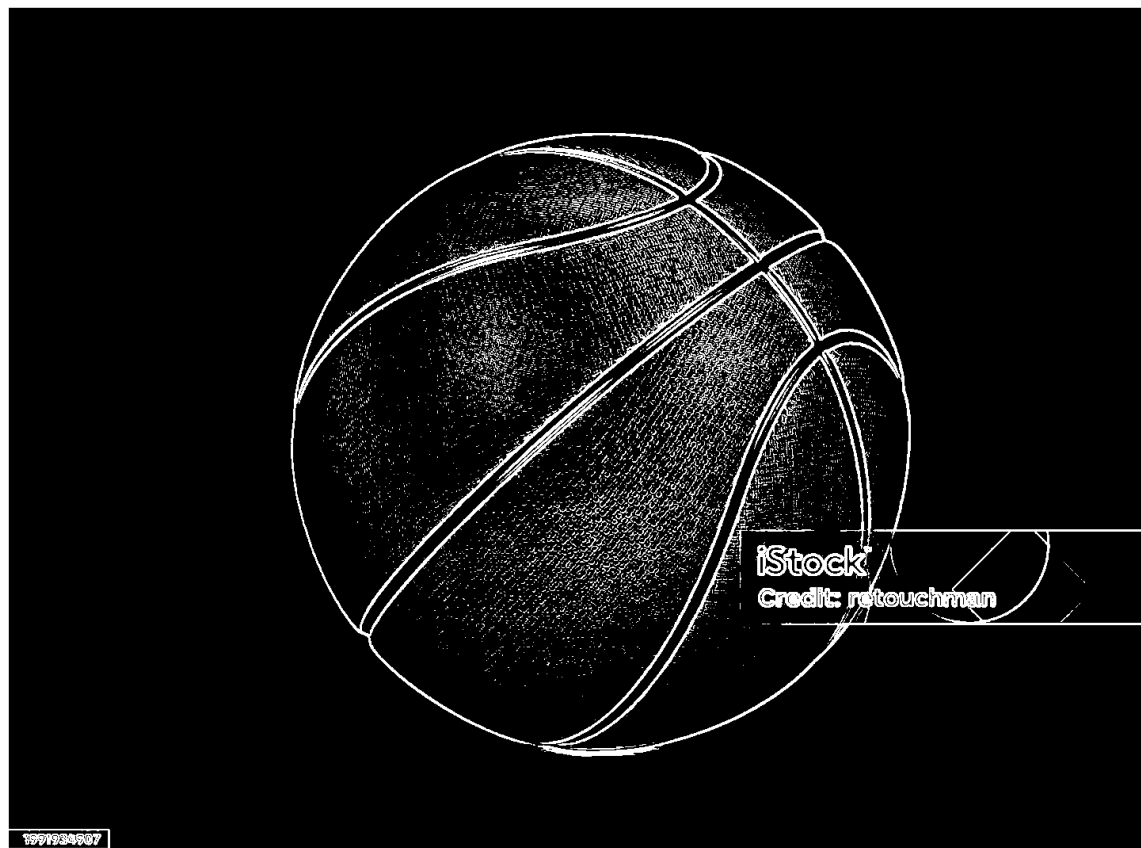


1991934907

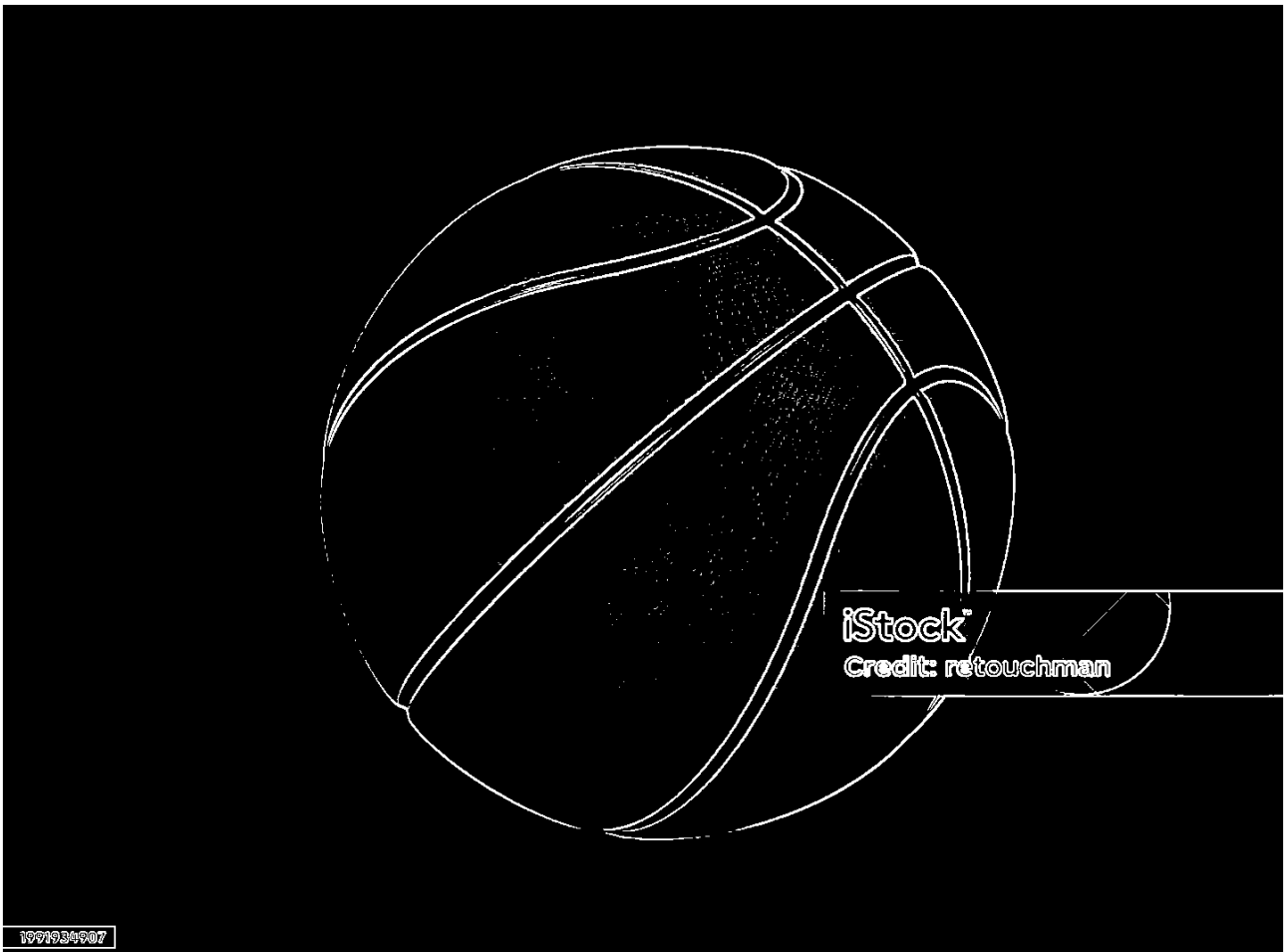
SOBEL



Thres: 0.1

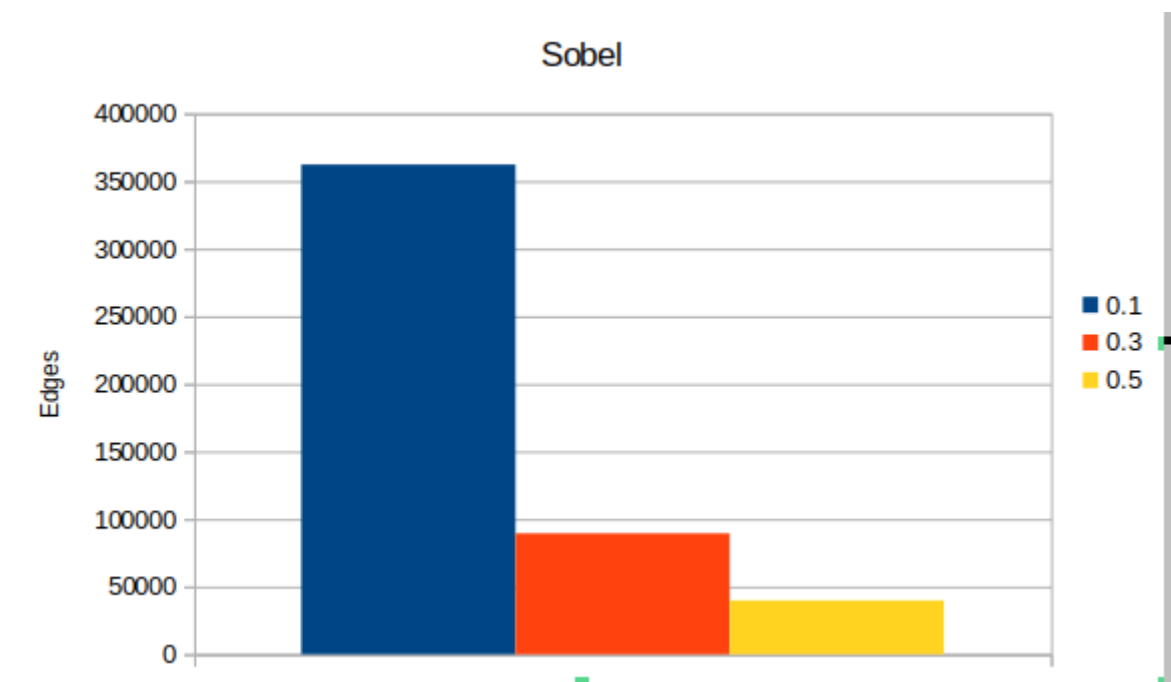


Thres: 0.3

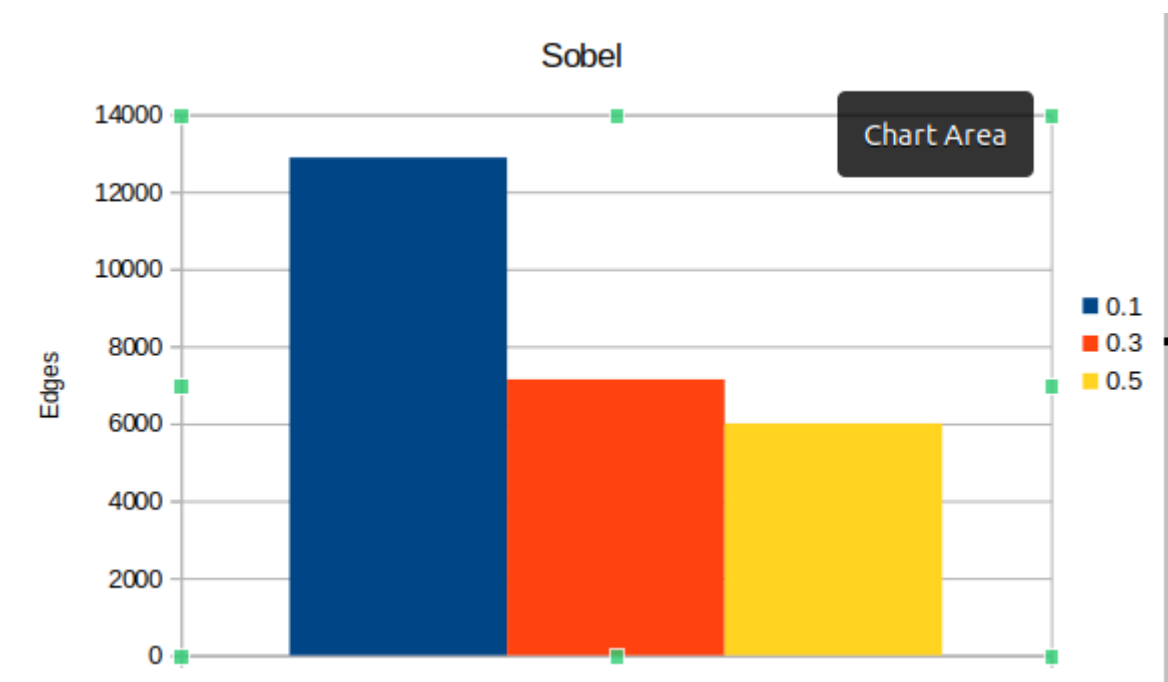


Thres: 0.5

ORIGINAL PIC



AFTER RESIZE

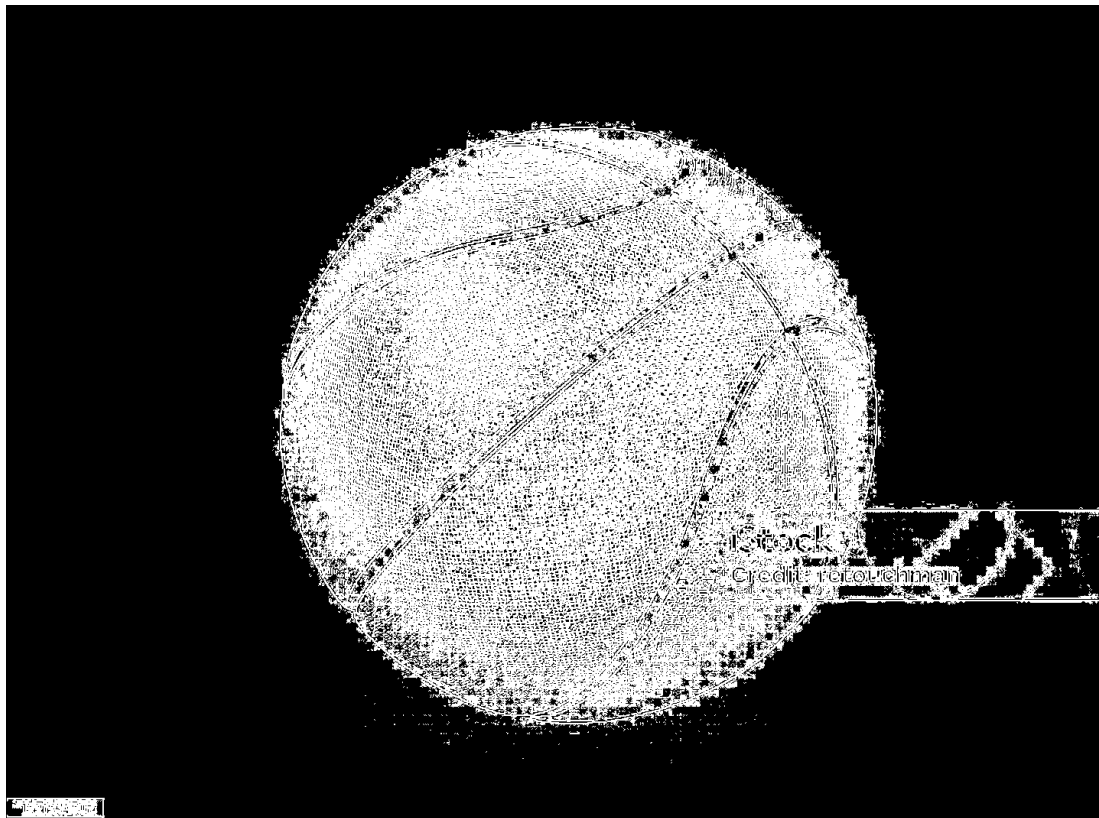


Βλέπουμε ότι για την μέθοδο Sobel παίρνουμε αρκετά καλά αποτελέσματα, οι γραμμές είναι ευδιάκριτες και δεν παίρνουμε πολλές “λανθασμένες” τιμές. Ιδιαίτερα όσο ανεβάζουμε το threshold βλέπουμε ότι τα λάθη μειώνονται και λαμβάνουμε περισσότερα σημεία πάνω στις γραμμές της μπάλας.

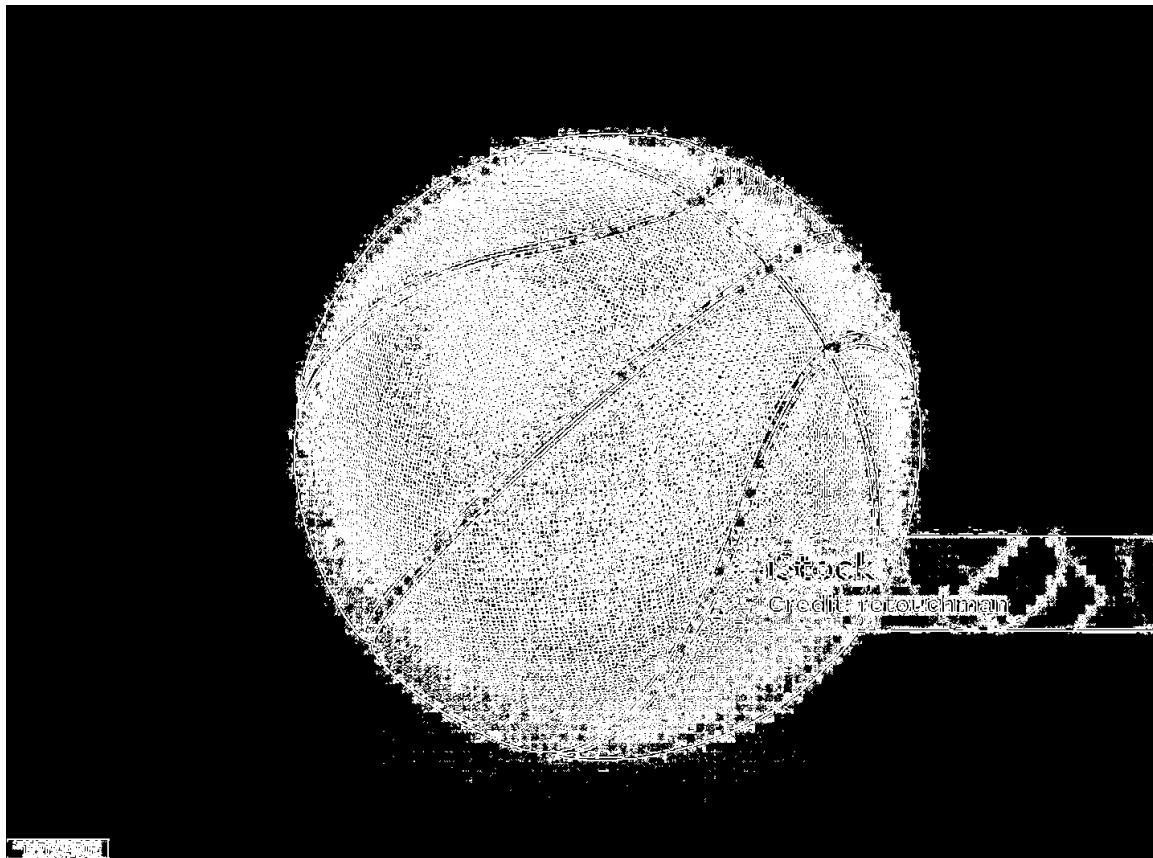
Το ίδιο συμπέρασμα βγάζουμε και από τα ιστογράμματα, καθώς βλέπουμε τη σαφή μείωση των λευκών σημείων για μεγαλύτερες τιμές του threshold.

Το δεύτερο σχεδιάγραμμα δείχνει τις ακμές που ανιχνεύτηκαν μετά τη σμίκρυνση της εικόνας, το οποίο παρουσιάζει παρόμοια αποτελέσματα σε μικρότερη κλίμακα.

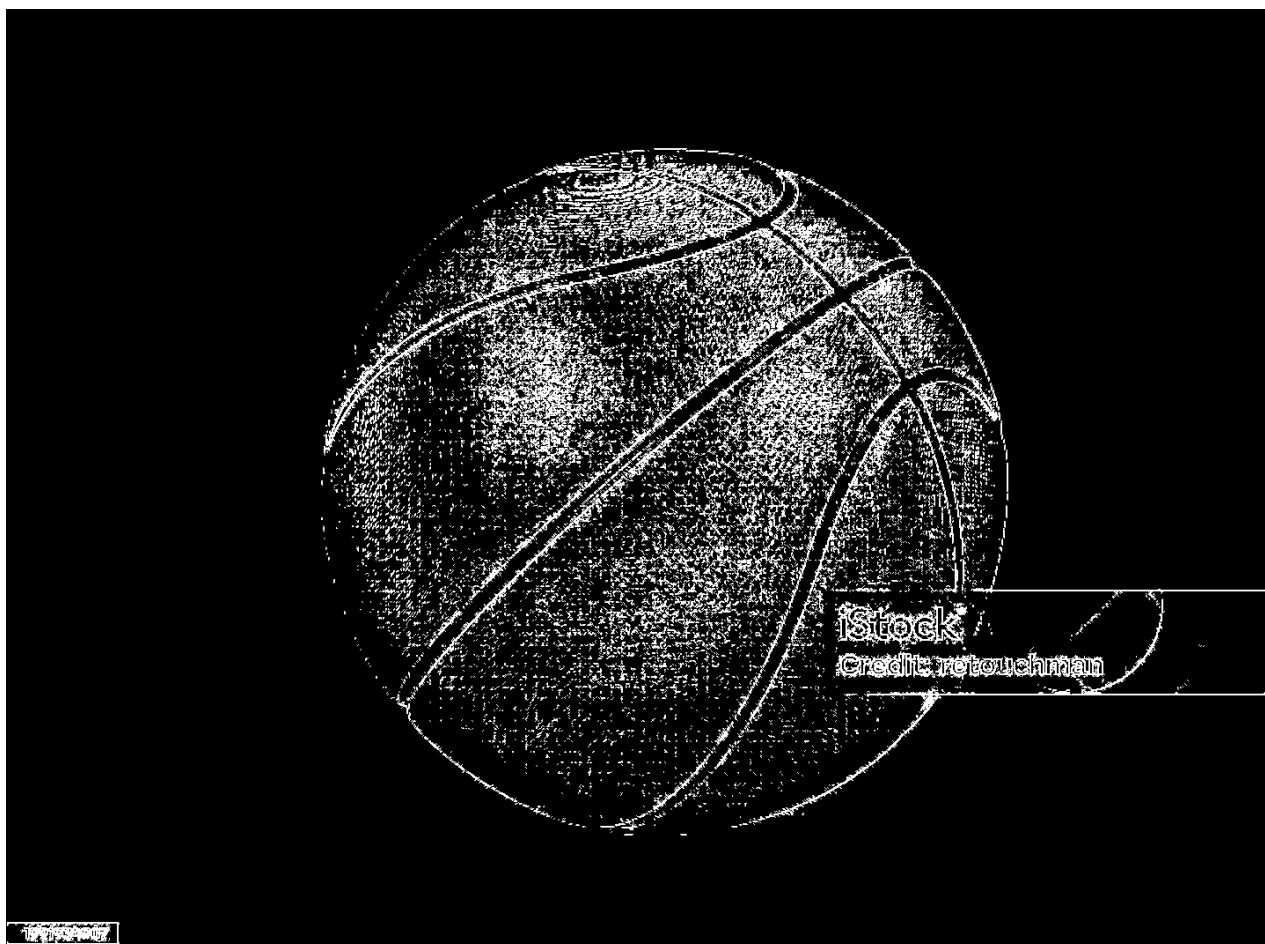
LoG



Min_dif: 0.01

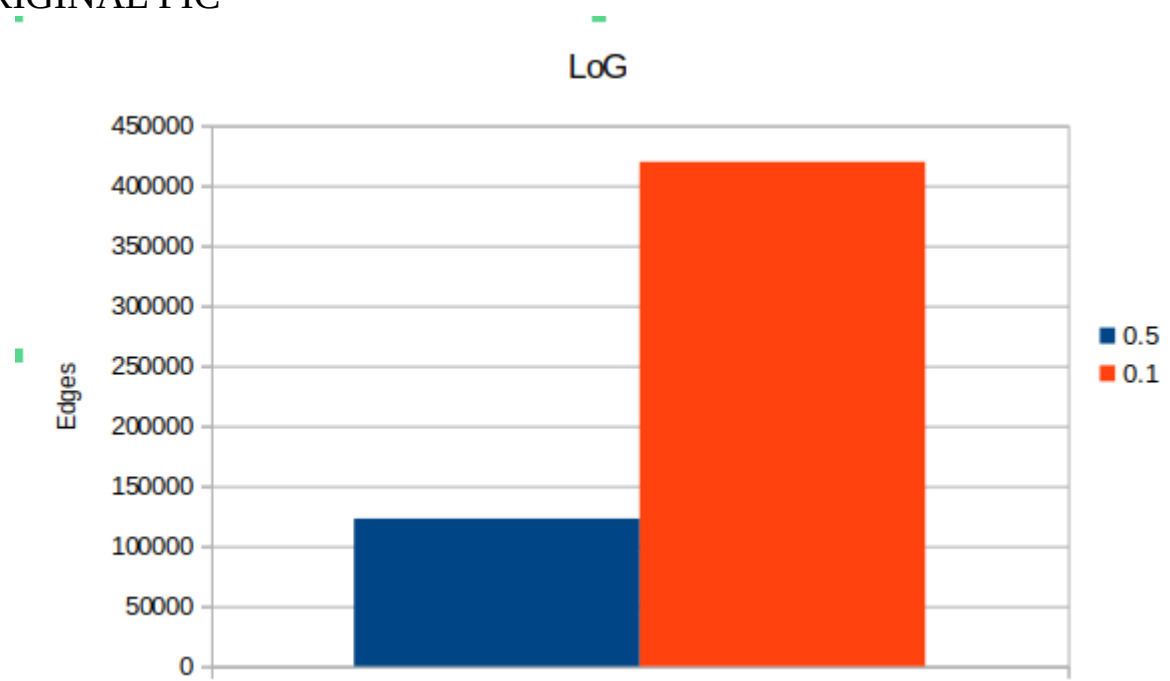


Min_dif: 0.1

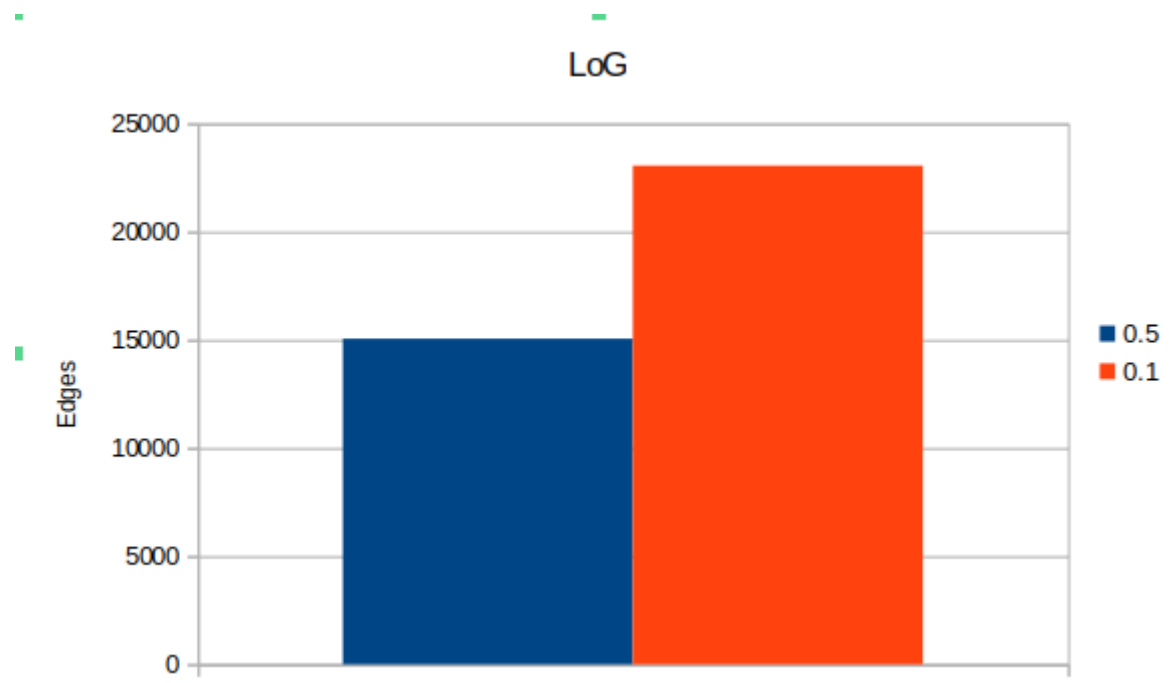


Min_dif: 0.5

ORIGINAL PIC



AFTER RESIZING



Τα αποτελέσματα για τη μέθοδο LoG είναι σαφώς χειρότερα, το οποίο συμβαίνει τόσο λόγω της μεθόδου, αλλά και λόγω του πίνακα που λάβαμε. Προστέθοντας και αυξάνοντας μια ελάχιστη διαφοράς μεταξύ των γειτονικών στοιχείων καταφέρνουμε να εξαλείψουμε μεγάλο κομμάτι του θορύβου, αλλά και πάλι δεν πλησιάζουμε τη μέθοδο Sobel, για τον συγκεκριμένο πίνακα. Τόσο στις εικόνες, όσο και στο διάγραμμα διακρίνουμε την τεράστια βελτίωση στην απόδοση με την αύξηση του `min_dif`.

HOUGH CIRCLES

Circles found with Hough



Sobel
0.3
Vmin 55
Rmin 60
Rmax 300
dim [100, 100, 50]

Circles found with Hough



Sobel
0.5
Vmin 50
Rmin 60
Rmax 300
dim [100, 100, 50]

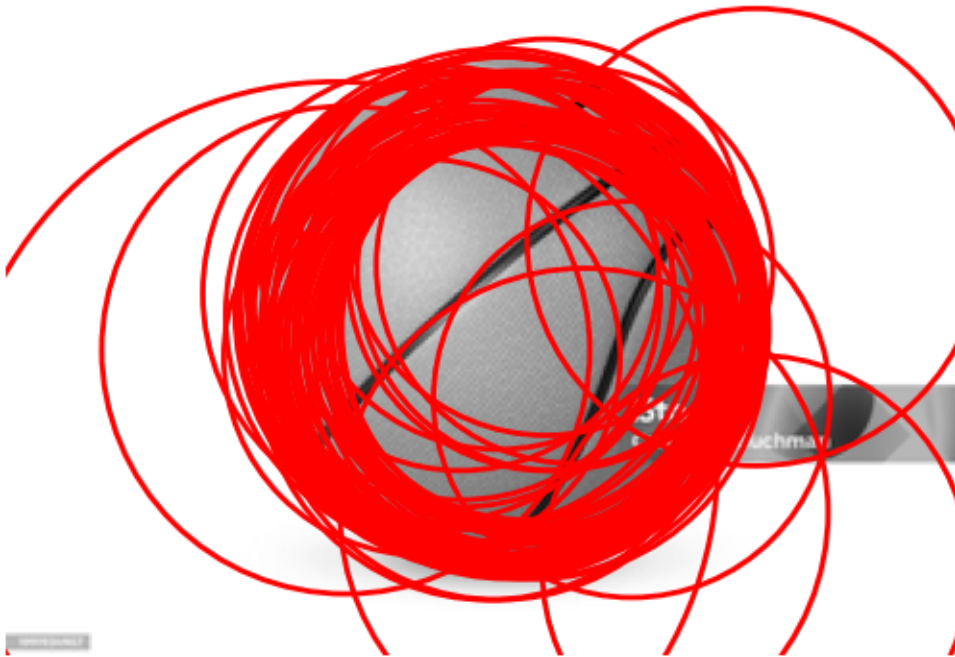
Circles found with Hough



Image courtesy of

Sobel
0.5
Vmin 40
Rmin 60
Rmax 300
dim [100, 100, 50]

Circles found with Hough



Sobel

0.5

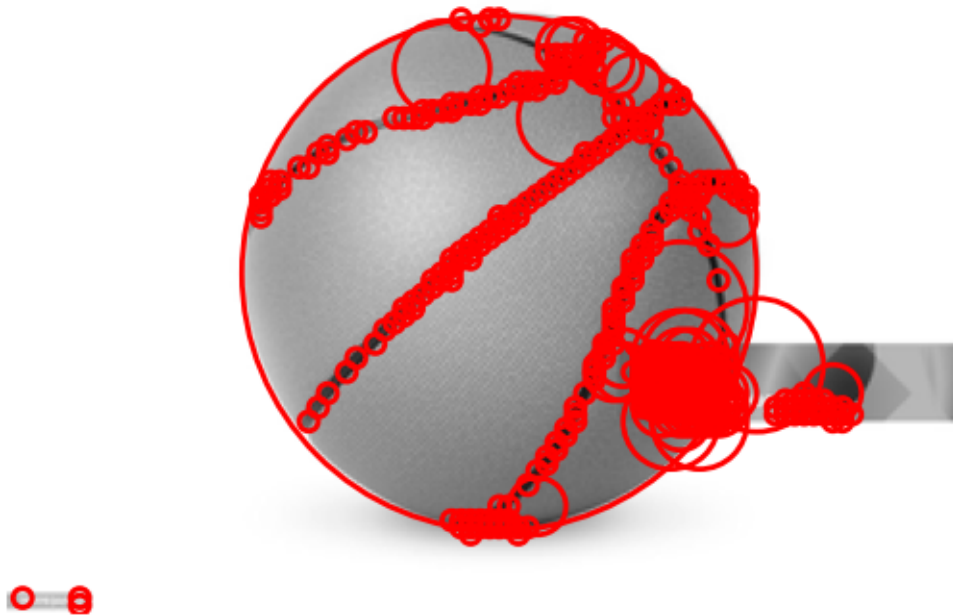
Vmin 30

Rmin 60

Rmax 300

dim [100, 100, 50]

Circles found with Hough



Sobel
0.5
Vmin 45
Rmin -
Rmax 300
dim [100, 100, 50]

Circles found with Hough



LoG
0.5
Vmin 55
Rmin 60
Rmax 300
dim [100, 100, 50]

Circles found with Hough



LoG
0.5
Vmin 65
Rmin 60
Rmax 300
dim [100, 100, 50]

Παρατηρήσεις

Όπως φαίνεται και στα αποτελέσματα παραπάνω πειραματιστηκα αρκετά με τις μεταβλητές της μεθόδου. Χρησιμοποιώντας το αποτέλεσμα της μεθόδου LoG, δεν κατάφερα να απομονώσω τον κύκλο της μπάλας για τις τιμές που δοκίμασα, το οποίο είναι και λογικό καθώς υπήρχαν πολλά “λάθος” σημεία.

Γενικότερα, διακρίθηκε το γεγονός των πολλών μικρών κύκλων στην αρχική υλοποίηση όπως φαίνεται και παραπάνω, το οποίο όμως εξαλείφθηκε με την προσθήκη του R_{min} .

Επίσης, όσον αφορά τον χρόνο εκτέλεσης σμικρύνουμε την εικόνα, όπως ήταν αναγκαίο καθώς η εκτέλεση δεν ολοκληρωνόταν και μειώθηκαν πολύ οι γωνίες ελέγχου, χωρίς όμως να επηρεάζει σε μεγάλο βαθμό τα αποτελέσματα.

Τέλος αυξάνοντας την “εμπιστοσύνη”, τον ελάχιστο αριθμό ψήφων για την δημιουργία κύκλου λάβαμε το επιθυμητό αποτέλεσμα τόσο για τιμή 0.5, όσο και για 0.3.